

User's manual for DISODE45, a Python integration code

M. Calvo ¹, Juan I. Montijano ² & Luis Rández ³

August 19, 2026

*Departamento Matemática Aplicada
Pza. San Francisco s/n, Universidad de Zaragoza.
50009-Zaragoza, Spain.*

¹ email: calvo@unizar.es

² email: monti@unizar.es

³ email: randez@unizar.es

1 The DISODE45 function

DISODE45 is a Python function that solves non-smooth, non-stiff differential systems

$$y'(t) = f(t, y(t)), \quad y(0) = y_0 \in \mathbb{R}^m, \quad t \in [t_0, t_f]$$

The non-smoothness of the solution $y = y(t)$ happens at points, switching points, (t_d, y_d) where one of several switching surfaces defined by a function $g(t, y) = (g_1(t, y), \dots, g_k(t, y))$ vanish, that is, at least for an index i , $g_i(t_d, y_d) = 0$.

The call to this function has any of the following possible syntax

```
T,Y = disode45(odefun,switchfun,tspan,y0)
T,Y,TDIS,YDIS,IDIS,STATS=disode45(odefun,switchfun,tspan,y0)
T,Y = disode45(odefun,switchfun,tspan,y0,options)
T,Y,TDIS,YDIS,IDIS,STATS=disode45(odefun,switchfun,tspan,y0,options)
sol=disode45(...)
```

It is very similar to the syntax used by the ODE suite Matlab package so that users that are familiar with this software can find the use of `disode45` very easy.

1.1 Input arguments

odefun A function that evaluates the right side of the differential equations $f(t, y)$.

switchfun A function that evaluates the function $g(t, y)$ defining the switching surfaces.

tspan A vector specifying the interval of integration, $[t_0, t_f]$. The solver imposes the initial conditions at **tspan**[0], and integrates from **tspan**[0] to **tspan**[-1]. At least, **tspan** must contain two points, the initial and final points of the integration interval. If **tspan** has more than two components, the integrator returns the approximation to the solution exactly at the points specified in **tspan**. If **tspan** has dimension two, the integrator returns the approximation to the solution at the internal points selected by the code. For the moment, the code does not allow negative evolution of the time, that is, **tspan** must contain an increasing set of time points.

y0 An m dimensional vector of initial conditions.

options Structure of optional parameters that change the default integration properties. You can create options using the **disodeset** function.

1.2 Output arguments

T Vector of time points.

Y Solution array. Each row in **Y** corresponds to the solution at a time returned in the corresponding component of **T**.

TDIS Vector of times at which discontinuities or switching points of the numerical solution have been detected.

YDIS The solution at the times of switching points. Each row in **YDIS** corresponds to the solution at a time returned in the corresponding component of **TDIS**.

IDIS Vector containing the indexes of the switching function that vanishes at each switching point.

- The absolute value of **IDIS**[*i*] indicates the switching function corresponding to the $(i + 1)$ -th switching point.
- The sign of **IDIS**[*i*] indicates the type of discontinuity. If it is positive, the discontinuity is transversal. If it is negative, the switching point is Filippov (entering or either exiting a sliding region). For example, a value **IDIS**[3] = -2 means that the fourth switching point is a Filippov point located at the second switching surface $g_2(t, y)$.

STATS Vector containing some statistics about the integration

STATS[0]	Number of accepted normal steps
STATS[1]	Number of rejected normal steps
STATS[2]	Number of accepted sliding steps
STATS[3]	Number of rejected sliding steps
STATS[4]	Number of steps at with a possible switching point
STATS[5]	Number of transversal discontinuities
STATS[6]	Number of sliding discontinuities
STATS[7]	Number of exits of a sliding region
STATS[8]	Number of calls to odefun
STATS[9]	Number of calls to switchfun
STATS[10]	Number of calls to gradswitchfun

sol Structure array containing the approximated solution and other relevant information. The sol structure array always includes these fields:

- sol.x containing the vector of time points T
- sol.y containing the approximated solution Y
- sol.xdis containing the vector of time points TDIS where discontinuities appear
- sol.ydis containing the approximated solution at the discontinuity points, YDIS
- sol.idis containing the vector of indices of the switching functions IDIS
- sol.stats containing the vector of statistics STATS

1.3 Required user's functions

odefun The function,

`odefun(t,y)`

for a scalar t and a column vector y , must return a column vector f corresponding to $f(t, y)$.

switchfun The function

`switchfun(t,y)`

for a scalar t and a column vector y , must return three column vectors

- **value[i]** is the value of the $(i+1)$ -th component $g_{i+1}(t, y)$
- **isterminal[i]** specifies the action to be taken when a zero of the function $g_{i+1}(t, y)$ is found. It can take the values -1 , 0 or 1 .
 - **isterminal[i]=1** means that the integration must terminate at a zero of the $(i + 1)$ -th switching function.

- `isterminal[i]=0` means that the integration must continue without any action.
- `isterminal[i]=-1` means that the program must call an external function `actionatswitch(t,y)`, that should have been provided by the user in the options by means of `disodeset` function (see sections 3 and 4).
- `direction[i]` specifies which zeros of $g_i(t, y)$ have to be computed.
 - `direction[i] = 0` means that all zeros are to be computed (the default).
 - `direction[i] = +1` means that only the zeros where the switching function increases, that is, it passes from negative to positive, must be computed.
 - `direction[i] = -1` means that only the zeros where the switching function decreases must be computed.

1.4 Getting the code

The code can be downloaded at <https://github.com/jimontijano/DISODE-Python>

2 A simple first example

Let us consider a simple mechanical system consisting on a mass $m = 1$ attached to a fixed wall by a spring with stiffness coefficient 1, moving on a surface with a Coulomb friction with friction coefficient μ so that the friction force is $F_C = \mu g = 0.4$.



This system is modelled by the non-smooth second order differential equation

$$x'' = -x - F_C \operatorname{sign}(x').$$

In order to be integrated by `DISODE45`, this second order equation must be expressed as a first order system with two components $y_1(t) = x(t)$ and $y_2(t) = x'(t)$, as

$$y' = \begin{pmatrix} y_1' \\ y_2' \end{pmatrix} = \begin{pmatrix} y_2 \\ -y_1 - F_C \operatorname{sign}(y_2) \end{pmatrix} = f(t, y)$$

Clearly the vector field is non smooth at points where $x' = y_2 = 0$. Then, we have a unique switching surface $g(y_1, y_2) = y_2$.

The solution, with initial conditions $x(0) = 3, x'(0) = 0$, passes first through 3 transversal discontinuities and after that, at $t = 12.5664 \dots$ it enters a sliding region with $x = 0.2, x' = 0$ and stays at this point forever (the friction force is greater than the force of the spring and the mass stops).

The evolution of the solution $x(t)$ and its derivative $x'(t)$ are depicted in Figure ???. The discontinuity points are indicated by means of small circles. Note that for this problem, since it is a second order differential equation, the solution $x(t)$ and its derivative $x'(t)$ are continuous, as it can be seen in the plots, but the second derivative $x''(t)$ is not continuous.

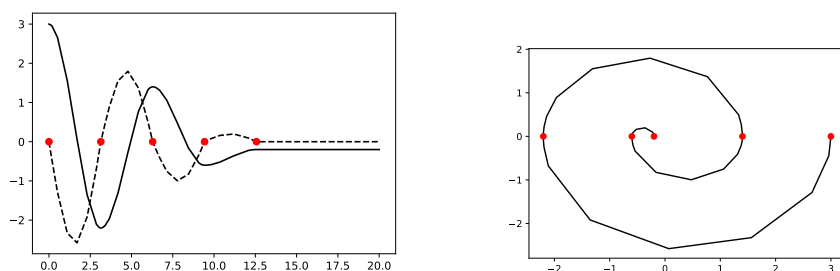


Figure 1: Left: Solution and derivative against time, Right: Phase diagram

A Python code to integrate this problem with `disode45` can be

```
import numpy as np
import matplotlib.pyplot as plt
from disode45 import *

#
# Definition of the vector field
#
def fun(t,y):
    Fc=0.4;
    f=np.array([y[1], -y[0]-Fc*np.sign(y[1])])
    return f

#
# Definition of the switching surface
#
def gfun(t,y):
    g=y[1]
    isterminal=0
    direction=0
    return g,isterminal,direction
```

```

#
# Call to disode45
#
y0=[3, 0]
tout,yout,tdis,ydis,idis,stats =disode45(fun, gfun,[0,20], y0)
#
# display of the results
#
print("\n tdis=",tdis)
print("\n ydis=",ydis)
print("\n idis=",idis)
#
# Plot of the figures
#
plt.figure(1)
plt.plot(tout,yout[:,0], 'k', tout,yout[:,1], 'k--', tdis,ydis[:,1], 'ro')

plt.figure(2)
plt.plot(yout[:,0],yout[:,1], 'k-', ydis[:,0],ydis[:,1], 'ro')

```

Once the integration of the problem has concluded, the vectors `tdis`, `ydis` and `idis` are displayed, giving

```

tdis= [ 0.          3.14160526  6.28321078  9.42482173 12.56648946]

ydis= [[ 3.00000000e+00  0.00000000e+00]
 [-2.20005193e+00 -7.45931095e-16]
 [ 1.40008785e+00  9.76649317e-16]
 [-6.00113635e-01  1.40859546e-15]
 [-1.99872225e-01  2.64545330e-16]]

idis= [1 1 1 1 1]

```

The elements of the vector `idis` indicate that all the switching points correspond to the first switching surface, an expected fact because there is a unique switching surface. The first four components are positive, which means that these discontinuities are transversal. The last one is negative, and therefore this discontinuity starts a sliding region. Note that the integration starts at an initial condition $(3,0)$ which belongs to the switching surface. This is why the vector `tdis` has 0 as its first element, which means that there is a switching point at $t = 0$.

The last two lines of the code generate the two plots contained in Figure 1. As it can be seen, the curves obtained look like a polygonal. This is due to the fact that the integration code uses 66 time steps, many of them around the second discontinuity point $(-2.20005193, 0)$. To get smoother curves, we can stete some options to be used by the code by means of the `disodeset` function.

3 The DISODESET function

DISODESET is a Python function that creates a options structure that lets the user set some parameters (options) to be used by DISODE45 in the numerical integration.

The call to this function has the following syntax

```
options=disodeset('name1',value1,'name2',value2,...)
```

3.1 Allowed options

'RelTol' Sets the Relative error tolerance (default to 1e-3). Each integration step satisfies $\|est\|_2 \leq RelTol * \|y\|_2 + AbsTol$.

'AbsTol' Sets the absolute error tolerance (default to 1e-6). See RelTol.

'InitialStep' Sets a suggested initial step size and must be a positive scalar. The solver will try this first. By default the solver determine an initial step size automatically.

'EventControl' Sets the type of control for the detection of discontinuities

- 0 Existence of discontinuity is checked at every step and every stage of failed steps.
- 1 Existence of discontinuity is checked at every stage of every step.
- k Existence of discontinuity is checked at every stage of every step and at $6*k$ uniformly distributed points inside every step.

'Refine' Sets the output refinement factor and must be a positive integer. This property increases the number of output points by the specified factor producing smoother output. Refine defaults to 4.

'Gradient' Specifies, by means of a function handle, the function to compute the gradient of the switching functions.

'GradientComponents' Specifies in a vector for which components (switching manifolds) of the switching function `switchfun` the gradient must be computed through the function specified in the option 'Gradient', and which ones must be computed by finite differences. A zero in the component i of the vector specified in 'GradientComponents' means that the gradient for the component i must be evaluated by finite differences and a 1 means that it must be evaluated using the external function. This option requires the option 'Gradient' to be set.

'ActionSwitch' Specifies, by means of a function handle, the function to be called by the integrator when a switching point is found. This output function is called if the corresponding value of the vector `isterminal` is -1 . ActionSwitch defaults to `[]`.

The plots in Figure ?? can be improved for example by setting

```
options = disodeset('RelTol',1.e-6, 'AbsTol',1.e-6)
tout,yout,tdis,ydis,idis,stats =disode45(fun, gfun,[0,20], y0, options)
```

This makes the program to integrate with a smaller error and therefore it takes more time steps. Another possibility is to force the code to compute the solution at more intermediate points with the attribute “Refine”

```
options = disodeset('Refine',10)
tout,yout,tdis,ydis,idis,stats =disode45(fun, gfun,[0,20], y0, options)

tspan=np.linspace(0,20,100)
tout,yout,tdis,ydis,idis,stats =disode45(fun, gfun,[0,20], y0)
```

With any of the above three possibilities we get the plots in Figure 2

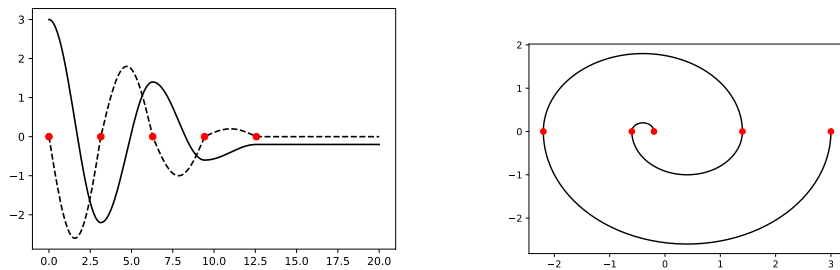


Figure 2: Left: Solution and derivative against time, Right: Phase diagram

4 Optional functions that can be provided by the user

gradswitchfun

In the analysis of the discontinuities, to determine if they are transversal or they start a sliding region, the code requires the computation of the gradient vector of the switching functions. By default, this is computed by finite differences. nevertheless, the user can provide an external function to compute it in an analytic way

```
grad = gradswitchfun(t,y,inddis)
```


For a scalar $\mathbf{t}$, column vector $\mathbf{y}$, and a switching function index $\mathbf{inddis}$, must return a column vector $\mathbf{grad}$ which is the gradient vector of the function $g_{\mathbf{inddis}}$

This function must be provided by the user if the option 'Gradient' is set. By default, `disode45` computes the gradient vector by means of divided differences.

In the example above, we can compute the gradient analytically by setting

```
options = disodeset('Refine',10, 'Gradswitchfun', gradfun)
tout,yout,tdis,ydis,idis,stats =disode45(fun, gfun,[0,20], y0, options)
```

actionatswitch The function

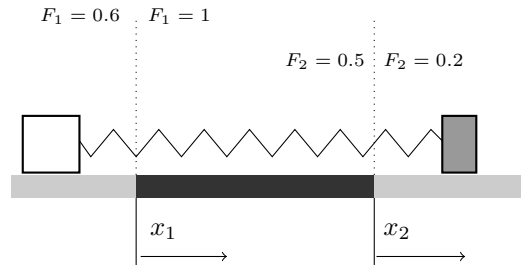
```
yout = actionatswitch(t,y)
```

for a scalar $\mathbf{t}$ and a column vector $\mathbf{y}$, must return a column vector $\mathbf{yout}$ which is the new state vector. The integration will proceed from the point t with the initial condition $\mathbf{yout}$.

This function must be provided if the option 'ActionSwitch' is set, and it is executed if the function `switchfun(t,y)` returns a value -1 in the vector `isterminal` at the switching point.

5 Examples

Example 1 In the first example we consider two masses $m_1 = m_2 = 1$ linked by a spring, moving on a surface with Coulomb friction (see [1, pp. 162]). Both masses are equal, but made with different material, so their friction coefficients μ_1 and μ_2 are different and so are the friction forces F_1 and F_2 . The surface where the masses move has two physically different parts, which makes the friction coefficients change.



This mechanical system is modelled by the non-smooth differential system

$$\begin{cases} x_1'' = -(x_1 - x_2) - F_1 \text{sign}(x_1'), \\ x_2'' = -(x_2 - x_1) - F_2 \text{sign}(x_2'), \\ F_1 = \begin{cases} 1 & \text{if } x_1 \leq 0 \\ 0.6 & \text{if } x_1 > 0 \end{cases} \quad F_2 = \begin{cases} 0.5 & \text{if } x_2 \leq 0 \\ 0.2 & \text{if } x_2 > 0 \end{cases} \\ x_1(0) = -2, \ x_2(0) = 3, \ x_1'(0) = 0, \ x_2'(0) = 0 \\ t \in [0, 6]. \end{cases}$$

Again, we must express the second order system as a first order system with four components $y_1(t) = x_1(t)$, $y_2(t) = x_2(t)$, $y_3(t) = x_1'(t)$ and $y_4(t) = x_2'(t)$

$$y' = \begin{pmatrix} y_1' \\ y_2' \\ y_3' \\ y_4' \end{pmatrix} = \begin{pmatrix} y_3 \\ y_4 \\ -(y_1 - y_2) - F_1 \text{sign}(y_3) \\ -(y_2 - y_1) - F_2 \text{sign}(y_4) \end{pmatrix} = f(t, y)$$

Clearly the vector field $f(t, y)$ is non smooth at points where $x_1' = y_3 = 0$, $x_2' = y_4 = 0$, $x_1 = y_1 = 0$ or $x_2 = y_2 = 0$. Then, we have four switching surfaces $g_1(y) = y_1$, $g_2(y) = y_2$, $g_3(y) = y_3$ and $g_4(y) = y_4$.

The solution, with these initial conditions, passes first through 14 transversal discontinuities and after some time it enters a sliding region onto $g_1(y) = 0$. Inside this sliding region, $y_1 = 0$, the solution crosses three times transversal discontinuities and finally it attains a co-dimension 2 sliding region $y_3 = y_4 = 0$, where both masses do not move, and the system remains there forever. The integrator stops when it find a co-dimension 2 Filippov point.

The components of the solution $x_1(t)$ and $x_2(t)$ of this problem as well as their derivatives are depicted in Figure 3. The 18 discontinuity points are indicated by means of small circles.

A Python code to integrate this problem with `disode45` can be

```
import matplotlib.pyplot as plt
from disode45 import *

#
# Definition of the vector field
#
def fun1(t,y):
    E=1.0
    if y[0] > 0:
        mu1=1.0
    else:
```

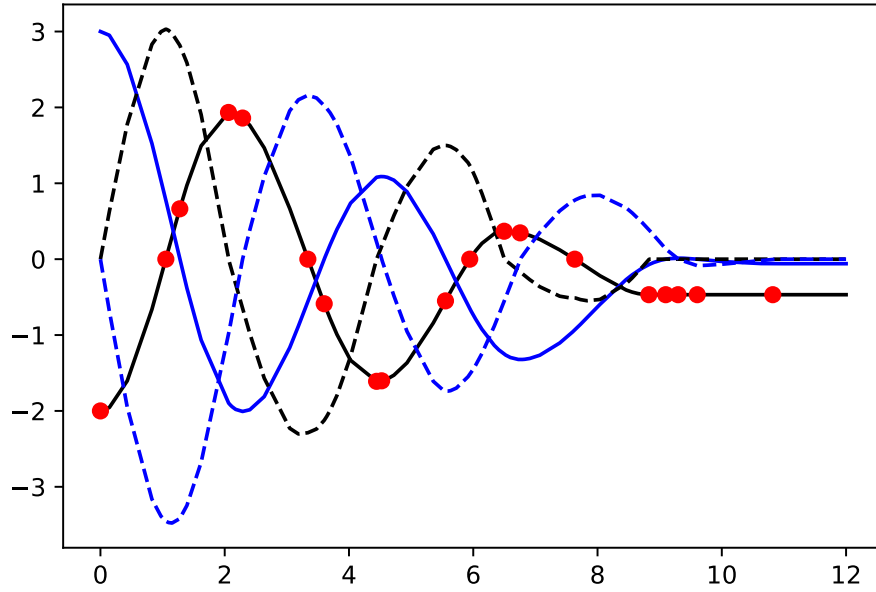


Figure 3: Solution for Example 1

```

mu1=0.6

if y[1] > 0:
    mu2=0.2
else:
    mu2=0.5

f=np.array([y[2],y[3],-E*(y[0]-y[1])-mu1*np.sign(y[2]),
            -E*(y[1]-y[0])-mu2*np.sign(y[3])])
return f

#
# Definition of the switching surface
#
def gfun1(t,y):
    g=np.array([y[2], y[3], y[0], y[1]])
    isterminal=[0,0,0,0]
    direction=[0,0,0,0]
    return g, isterminal,direction

```

```

#
# Call to disode45
#
y0 = np.array([-2, 3, 0, 0])
tout,yout,tdis,ydis,idis,stats=disode45(fun1, gfun1,[0, 12], y0)
#
# display of the results
#
print("\n tdis=",tdis)
print("\n idis=",idis)
#
# Plot of the figures
#

```

Once the integration of this problem has concluded, the vectors `tdis` and `idis` contain the following data

```

tdis= [ 0.          1.05248095  1.27729353  2.06110823
        2.28783783  3.33792416  3.6011638   4.44409678
        4.52562036  5.55298383  5.94326324  6.49709212
        6.7546499   7.63358362  8.82595478  9.09732769
        9.29548718  9.60450416 10.82038526]

idis= [ 1  3  4  1  2  3  4  1  2  4  3  1  2
        3 -1  4  2  4 -2]

```

The elements of the vector `idis` indicate that the first switching point correspond to the first switching surface, the second one to the third surface and so on. The positive elements mean that these discontinuities are transversal. The negative ones mean that the solution enters into a sliding region.

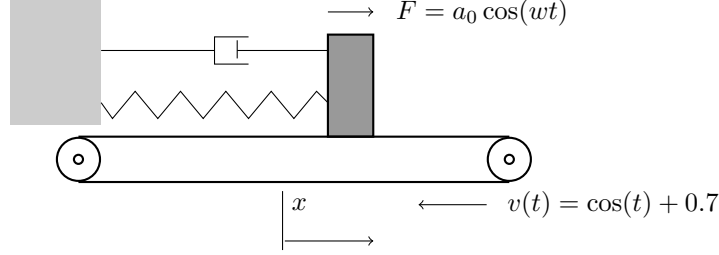
The plots in Figure 3 (without legends) can be obtained with the Python order

```

plt.plot(tout,yout[:,0], 'k', tout,yout[:,1], 'b', tout,yout[:,2], 'k--',
         tout,yout[:,3], 'b--', tdis,ydis[:,0], 'ro')

```

Example 2 In the second example (see [4]) we consider a mass $m = 1$ linked to a wall by a spring of stiffness k and a damper of viscous damping coefficient r . A external force $F = a_0 \cos(\omega t)$ is acting upon the mass which is placed onto a belt that moves with velocity $v(t) = \cos(t) + 0.7$. Therefore, a Coulomb friction force F_C acts upon the mass.



This mechanical system is modelled by the non-smooth second order differential system

$$\begin{cases} x'' = -kx - 2rx' + a_0 \cos(wt) - F_C \operatorname{sign}(x' - v(t)), \\ F_C = 0.4 \quad a_0 = 1, \quad r = 0.2, \quad k = 1, \quad w = 0.7, \\ v(t) = \cos(t) + 0.7, \\ x(0) = 3, x'(0) = 0, \\ t \in [0, 30]. \end{cases}$$

Expressed as a first order system with two components $y_1(t) = x(t)$, $y_2(t) = x'(t)$, we have

$$y' = \begin{pmatrix} y_1' \\ y_2' \end{pmatrix} = \begin{pmatrix} y_2 \\ -ky_1 - 2ry_2 + a_0 \cos(wt) - F_C \operatorname{sign}(y_2 - v(t)) \end{pmatrix} = f(t, y)$$

Clearly the vector field $f(t, y)$ is non smooth at points where $x'(t) = y_2(t) = v(t)$. Then, we have one switching surface $g(y) = y_2 - v(t)$, that depends on y_2 and also on $v(t)$.

The solution $x(t)$ and its derivative $x'(t)$ are depicted in the upper plot of Figure 4. The discontinuity points (where the second derivative $x''(t)$ is not continuous) are indicated by means of small circles. To visualize the sliding regions, we give in the down plot of Figure 4 the function $x'(t) - v(t)$. The sliding regions correspond to the intervals at which the functions vanishes.

The solution, with the considered initial conditions, passes first through a transversal discontinuity, then it enters a sliding region for a short time until it exits it, then it passes through two transversal discontinuities and then it enters into another sliding region.

A Python code to integrate this problem with `disode45` can be

```
import numpy as np
import matplotlib.pyplot as plt
from disode45 import *
```

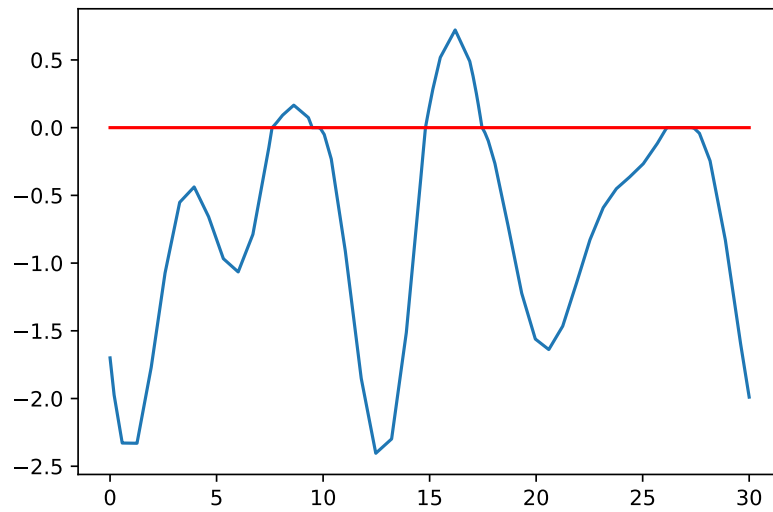
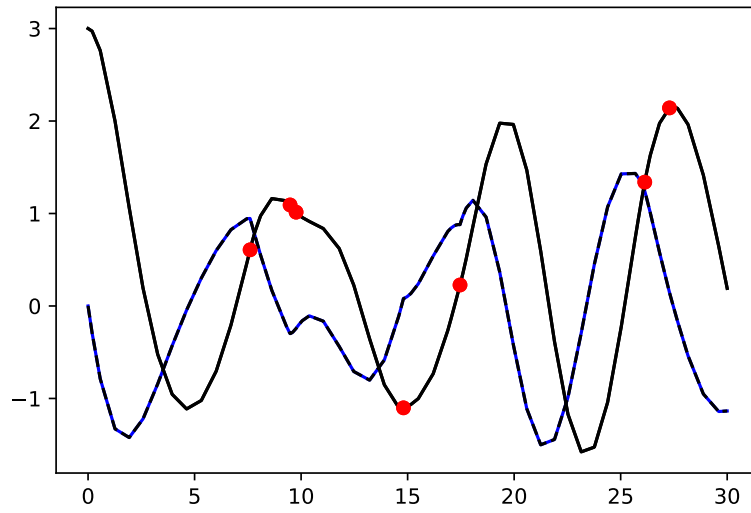


Figure 4: Example 2, Solution (top) and Sliding regions (down)

```
#
# Definition of the vector field
#
```

```

def fun(t,y):
    k=1.0
    r=0.2
    a0=1.0
    w=0.7
    Fc=0.4
    v=np.cos(t)+0.7
    f=np.array([y[1],-k*y[0]-2*r*y[1]+a0*np.cos(w*t)-Fc*np.sign(y[1]-v)])
    return f

#
# Definition of the switching surface
#
def gfun(t,y):
    v=np.cos(t)+0.7;
    g=y[1]-v;
    isterminal=0;
    direction=0;
    return g,isterminal,direction

#
# Call to disode45
#
y0 = np.array([3, 0])
tout,yout,tdis,ydis,idis,stats=disode45(fun, gfun,[0, 30], y0)
#
# display of the results
#
print("\n tdis=",tdis)
print("\n ydis=",ydis)
print("\n idis=",idis)

```

Once the integration of this problem concludes, the vectors tdis, ydis and idis contain the following data

```

tdis= [ 7.60556882  9.48332918  9.76525485 14.80509878 17.45896505
        26.13166544 27.2910479 ]

```

```

ydis= [
[ 0.6072464  0.9458658 ]
[ 1.09239123 -0.29828637]
[ 1.01432032 -0.24259552]
[-1.10142325  0.08063641]
[ 0.22699373  0.8792317 ]
[ 1.3389081  1.24120724]
[ 2.1419099  0.14570948]]

```

```
idis= [ 1 -1 -1  1  1 -1 -1]
```

In this case, the second element of `idis` equal to -1 means that the solution enters into a sliding region, and the third one, also -1 , means that the solution exits from the sliding region. The same happens with the last two elements.

The plots in Figure 4 (without legends) can be obtained with the Python orders

```
plt.figure(1)
plt.plot(tout,yout[:,0], 'k', tout,yout[:,1], 'b',
         tdis,ydis[:,0], 'ro')
plt.plot(tout,yout[:,0], 'k', tout,yout[:,1], 'k--', tdis,
         ydis[:,0], 'ro')

plt.figure(2)
plt.plot(tout,yout[:,1]-0.7*np.cos(tout), [0,30], [0,0], 'r')
plt.show()
```

Example 3 In the third example (see A. Luo [4, pp. 115]) we consider two masses m_1 and m_2 linked to a wall by springs of stiffness k_1, k_2 respectively and dampers of viscous damping coefficients r_1, r_2 . A external force $F = a_0 \cos(wt)$ is acting upon the mass m_1 . Mass m_1 is placed into a hole in mass m_2 so that the masses can hit each other as depicted in Figure 5.

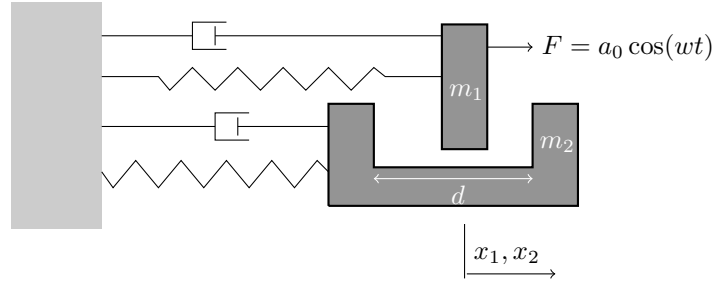


Figure 5: Example 3

Whenever both masses have no contact, that is, $y_1 - y_2 > d/2$ or $y_2 - y_1 > d/2$, this mechanical system is modelled by the differential system

$$\begin{aligned} m_1 x_1'' &= -k_1 x_1 - r_1 x_1' + a_0 \cos(wt) + b_0 \\ m_2 x_2'' &= -k_2 x_2 - r_2 x_2' \end{aligned} \quad (1)$$

When the masses hit each other, $|x_1 - x_2| = d/2$, the behaviour of the system depends on their velocity. If they are different, $x_1' \neq x_2'$, we have

a switching point in which the vector field does not change but the initial conditions of the system change from (x_1, x_2, x'_1, x'_2) to $(\hat{x}_1, \hat{x}_2, \hat{x}'_1, \hat{x}'_2)$ according to the following formulas

$$\begin{aligned}\hat{x}_1 &= x_1, \\ \hat{x}_2 &= x_2, \\ \hat{x}'_1 &= \frac{m_1 - m_2 e}{m_1 + m_2} x'_1 + \frac{(1 + e)m_2}{m_1 + m_2} x'_2, \\ \hat{x}'_2 &= \frac{m_2 - m_1 e}{m_1 + m_2} x'_1 + \frac{(1 + e)m_1}{m_1 + m_2} x'_2.\end{aligned}$$

If the masses hit with the same velocity, the system enters into a sticking region that is governed by the differential system

$$\begin{aligned}(m_1 + m_2)x''_1 &= -(k_1 + k_2)x_1 - (r_1 + r_2)x'_1 + \frac{k_2 d}{2} + a_0 \cos(wt) + b_0, \\ (m_1 + m_2)x''_2 &= -(k_1 + k_2)x_2 - (r_1 + r_2)x'_2 - \frac{k_1 d}{2} + a_0 \cos(wt) + b_0,\end{aligned}$$

if $x_1 = x_2 + d/2$ or

$$\begin{aligned}(m_1 + m_2)x''_1 &= -(k_1 + k_2)x_1 - (r_1 + r_2)x'_1 - \frac{k_2 d}{2} + a_0 \cos(wt) + b_0, \\ (m_1 + m_2)x''_2 &= -(k_1 + k_2)x_2 - (r_1 + r_2)x'_2 + \frac{k_1 d}{2} + a_0 \cos(wt) + b_0,\end{aligned}$$

if $x_2 = x_1 + d/2$.

Note that in this region, since $|x_1 - x_2| = d/2$, it holds $x''_1(t) = x''_2(t)$ and $x'_1(t) = x'_2(t)$. The system stays in this region while the forces per unit mass, that act upon each mass

$$\begin{aligned}u_1 &= \frac{F_1}{m_1} = -\frac{r_1}{m_1}x'_1 - \frac{k_1}{m_1}x_1 + \frac{b_0 + a_0 \cos(wt)}{m_1} \\ u_2 &= \frac{F_2}{m_2} = -\frac{r_2}{m_2}x'_2 - \frac{k_2}{m_2}x_2\end{aligned}$$

satisfy $x_1 - x_2 = d/2$ and $u_1 \geq u_2$ or well $x_2 - x_1 = d/2$ and $u_2 \geq u_1$. In other case, the systems goes back to the non sticking evolution and equations (1) govern the evolution of the system.

With the above considerations, and defining the state vector by $y(t) = (y_1(t), y_2(t), y_3(t), y_4(t))^T = (x_1(t), x_2(t), x'_1(t), x'_2(t))^T$ the system can be modelled by the non smooth system

$$y'(t) = A(t)y(t) + b(t),$$

with

$$A(t) = \begin{cases} \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -\frac{k_1+k_2}{m_1+m_2} & -\frac{r_1+r_2}{m_1+m_2} & 0 & 0 \\ -\frac{k_1+k_2}{m_1+m_2} & -\frac{r_1+r_2}{m_1+m_2} & 0 & 0 \end{pmatrix}, & \begin{array}{l} \text{if } y_1 - y_2 \geq d/2 \text{ and } u_1 \geq u_2 \\ \text{or } y_2 - y_1 \geq d/2 \text{ and } u_2 \geq u_1 \end{array} \\ \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -\frac{k_1}{m_1} & -\frac{r_1}{m_1} & 0 & 0 \\ -\frac{k_2}{m_2} & -\frac{r_2}{m_2} & 0 & 0 \end{pmatrix}, & \text{otherwise.} \end{cases}$$

$$b(t) = \begin{cases} \begin{pmatrix} 0 \\ 0 \\ \frac{k_2 d/2}{m_1+m_2} + \frac{a_0 \cos(wt)}{m_1+m_2} \\ \frac{-k_1 d/2}{m_1+m_2} + \frac{a_0 \cos(wt)}{m_1+m_2} \end{pmatrix}, & \text{if } y_1 - y_2 \geq d/2 \text{ and } u_1 \geq u_2 \\ \begin{pmatrix} 0 \\ 0 \\ \frac{-k_2 d/2}{m_1+m_2} + \frac{a_0 \cos(wt)}{m_1+m_2} \\ \frac{k_1 d/2}{m_1+m_2} + \frac{a_0 \cos(wt)}{m_1+m_2} \end{pmatrix}, & \text{if } y_2 - y_1 \geq d/2 \text{ and } u_2 \geq u_1 \\ \begin{pmatrix} 0 \\ 0 \\ \frac{b_0}{m_1} + \frac{a_0}{m_1} \cos(wt) \\ 0 \end{pmatrix}, & \text{otherwise} \end{cases}$$

In this example we have taken

$$r_1 = 0.6, \quad r_2 = 0.6, \quad k_1 = 30, \quad k_2 = 20, \quad a_0 = 1, \quad w = 0.7, \quad b_0 = 35$$

and as initial conditions

$$y_1(0) = 0.2, y_2(0) = 0.3, y_1'(0) = 0, y_2'(0) = 0, \quad t \in [0, 10].$$

The vector field $f(t, y)$ is non smooth at points where $y_1 - y_2 = d/2$, when $y_2 - y_1 = d/2$. Starting with the selected initial conditions, the system evolves smoothly until the masses hit. Then, a series of hits happen repeatedly, each time after less time. After a finite time, but infinite hits, the masses hit with the same velocity and the system enters into a sticking region. After some time, the system leaves this region and the behaviour repeats the same process. In the bottom plot of Figure 6 we depict the function $x_1(t) - x_2(t)$. The points where $x_1(t) - x_2(t) = 0.5$ correspond to points where the masses hit, and the intervals where $x_1(t) - x_2(t)$ correspond to sticking regions. In the top plot of Figure 6 we show the solution $x_1(t), x_2(t)$ against time t . again, the discontinuity points are indicated by means of small circles.

This problem has two specially complicated characteristics. On one side, the initial point of a sticking region is in fact a accumulation point of discontinuities, and in theory to attain it we should detect an infinity number of switching points. To solve this, the start of the sticking region can be assumed when $|x_1 - x_2| = d/2$ and $|x'_1(t) - x'_2(t)| < c$ for some small value of c . In our numerical computations we ha taken $c = 10^{-6}$. The second special characteristic is that at the moment the sticking region is attained the trajectory of the system is tangent to the switching surface. Thus, for the surface $g_1(y) = y_1 - y_2 - d/2 = 0$, the gradient vector is $\nabla g_1 = (1, -1, 0, 0)^T$ and the sticking region is attained at a point $y^* = (y_2 + d/2, y_2, y_4, y_4)^T$. At this point, the vector field has a value $f(t, y^*) = (y_4, y_4, -(k_1/m_1)y_1, -(k_2/m_2)/y_2)^T$ and clearly, $\nabla g_1(y^*) \cdot f(t, y^*) = 0$. This means that the numerical approximation can not be reliable, in the sense that a small error in the approximation of the solution can lead to a great error in the time t at which the solution starts the sticking region.

A Python code to integrate this problem with `disode45` can be

```
import numpy as np
import matplotlib.pyplot as plt
from disode45 import *

#
# Definition of the vector field
#
def fun(t,y):
    m1=2
    m2=1
    r1=0.6
    r2=0.6
    k1=30;
    k2=20;
    a0=30;
```

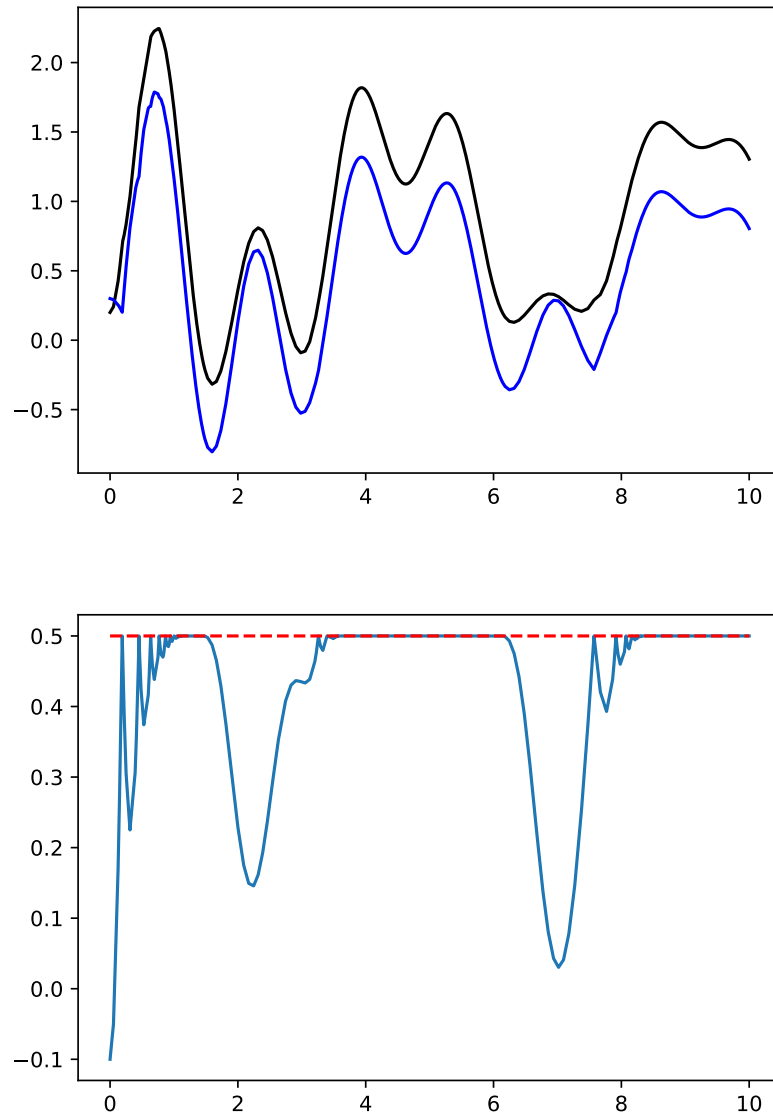


Figure 6: Example 3, Solution (top) and Sliding regions (bottom)

```

b0=35;
d=1.0;
ee=0.7
w=1.38

```

```

u1=-(r1/m1)*y[2]-(k1/m1)*y[0]+b0/m1+(a0/m1)*np.cos(w*t);
u2=-(r2/m2)*y[3]-(k2/m2)*y[1];
if y[0]-y[1]>= d/2 and u1>=u2: # and y[2] - y[3] >= -1e-4
    aux = -((r1 + r2) / (m1 + m2)) * y[2] - ((k1 + k2) / (m1 + m2)) * y[0]
    + b0 / (m1 + m2) + (k2 * d / (2 * m1 + 2 * m2))
    + (a0 / (m1 + m2)) * np.cos(w * t)
    f=np.array([y[2], y[2], aux, aux])
elif y[1]-y[0]>= d/2 and u2>=u1: # and Y[2] - Y[3] <= 1e-4
    aux = -((r1 + r2) / (m1 + m2)) * y[2] - ((k1 + k2) / (m1 + m2)) * y[0]
    + b0 / (m1 + m2) - (k2 * d / (2 * m1 + 2 * m2))
    + (a0 / (m1 + m2)) * np.cos(w * t)
    f=np.array([y[2], y[2], aux, aux])
else:
    f=np.array([y[2], y[3], u1, u2])

    return f

#
# Definition of the switching surface
#
def gfun(t,y):
    m1=2;
    m2=1;
    r1=0.6;
    r2=0.6;
    k1=30;
    k2=20;
    a0=30;
    b0=35;
    d=1.0;
    ee=0.7;
    w=1.38;
    u1=-(r1/m1)*y[2]-(k1/m1)*y[0]+b0/m1+(a0/m1)*np.cos(w*t);
    u2=-(r2/m2)*y[3]-(k2/m2)*y[1];

    if np.abs(y[0] - y[1] - d / 2) < 1e-12:
        isterminal=np.array([-1, -1, -1])
        direction=np.array([1, 1, -1])
        g = np.array([(y[0] - y[1] - d / 2), y[1] - y[0] - d / 2, u1 - u2])
    elif np.abs(y[1] - y[0] - d / 2) < 1e-12:
        isterminal=np.array([-1, -1, 1])
        direction=np.array([1, 1, 1])
        g = np.array([(y[0] - y[1] - d / 2), y[1] - y[0] - d / 2, u1 - u2])
    else:
        isterminal = np.array([-1, -1, 0])
        direction = np.array([1, 1, 0])
        g = np.array([(y[0] - y[1] - d / 2), y[1] - y[0] - d / 2, 1.0])

```

```

        return g, isterminal, direction

#
# Definition of the action at switch function
#
def actionatswitch(t,y):
    m1=2
    m2=1
    r1=0.6
    r2=0.6
    k1=30
    k2=20
    a0=30
    b0=35
    d=1.0
    ee=0.7
    w=1.38
    ysw=y.copy()
    ysw[2]=((m1-m2*ee)/(m1+m2))*y[2]+((1+ee)*m2/(m1+m2))*y[3]
    ysw[3]=((m2-m1*ee)/(m1+m2))*y[3]+((1+ee)*m1/(m1+m2))*y[2]

    if abs(y[0]-y[1]-d/2) <1.e-5:
        ysw[0]=y[1]+d/2-1.e-8;
    elif abs(y[1]-y[0]-d/2)<1.e-5:
        ysw[1]=y[0]+d/2-1.e-8;
    if abs(y[2]-y[3])<1.e-5:
        ysw[3]=y[3]
        ysw[2]=ysw[3]

    return ysw

#
# Call to disode45
#

options=disodeset('RelTol',1.e-5,'AbsTol',1.e-5,
                  'ActionSwitch', actionatswitch)
y0 = np.array([0.2,0.3,0,0])
[tout,yout,tdis,ydis,idis,stats]=disode45(fun, gfun,
                                           [0,10], y0,options);

```

Once the integration of this problem concludes, the vectors `tdis`, `ydis` and `idis` contain the data corresponding to 242084 discontinuity points. The plots in Figure 6 (without legends) can be obtained with the Python

orders

```
#
# Plot of the figures
#
plt.figure(1)
plt.plot(tout,yout[:,0], 'k', tout,yout[:,1], 'b')
plt.figure(2)
plt.plot(tout,yout[:,0]-yout[:,1], [0,10], [0.5,0.5], 'r--')
plt.show()
```

Example 4 In this example, we consider a simplified model of structural pounding, used in the study of the effects of earthquakes [3]. It is defined by the second order equation

$$2x'' = -4.1 x' - 210.125 x - u(x, x') - r(t), \quad t \in [0, 3]$$

with $r(t) = 2 \sin(14 t)$ and u given by

$$u(y, y') = \begin{cases} 0 & \text{if } x < \nu, \\ c \cdot (x - \nu)^{\frac{3}{2}} + 1.98\sqrt{2c}(x - \nu)^{\frac{1}{4}} x' & \text{if } x > \nu, x' > 0, \\ c \cdot (x - \nu)^{\frac{3}{2}} & \text{if } x > \nu, x' < 0, \\ c = 2.47 \times 10^6, \quad \nu = 0.005. \end{cases}$$

Expressed as a first order system with two components $y_1(t) = x(t)$, $y_2(t) = x'(t)$, we have

$$y' = \begin{pmatrix} y_1' \\ y_2' \end{pmatrix} = \begin{pmatrix} y_2 \\ -4.1 y_2 - 210.125 y_1 - u(y_1, y_2') - r(t) \end{pmatrix} = f(t, y)$$

Clearly the vector field $f(t, y)$ is non smooth at points where $x(t) = y_1(t) = \nu$ or where $x(t) = y_1(t) > \nu$ and $x'(t) = y_2(t) = 0$. Then, we have two switching surfaces $g_1(y) = y_1 - \nu$ and $g_2(y) = y_2$. For the switching surface g_2 , the vector field is discontinuous only when x' changes from positive to negative, which happens when $x > \nu$. Moreover, due to the powers $1/4$ and $3/2$, the function defining the vector field at the region $g_1(y) > 0$ is not defined when $g_1(y) < 0$.

Note that the vector field f is a continuous function (but not $\mathcal{C}^1$). Therefore $f_+(t_d, y_d) = f_-(t_d, y_d)$ at the switching points and the transversality condition is satisfied unless the vector field is tangent to the switching surface. It is easy to see that $\nabla g_1(y) \cdot f(t, y) = 1$ for all y and $\nabla g_2 \cdot f(t, y) = -210.125 y_1 - c \cdot (y_1 - \nu)^{3/2} - r(t)$ for switching points

such that $y_2 = 0$. Consequently, the transversality condition is satisfied except for the points for which $y_2 = x' = 0$, $y_1 = x > \nu$ and

$$-210.125 x(t) - c (x(t) - \nu)^{3/2} - r(t) = 0.$$

Since $|r(t)| \leq 2$, whenever $x(t) \notin [\nu, 0.0051157248]$ the discontinuity points are transversal.

The phase diagram for this problem (x_1 versus x') is depicted in Figure 7

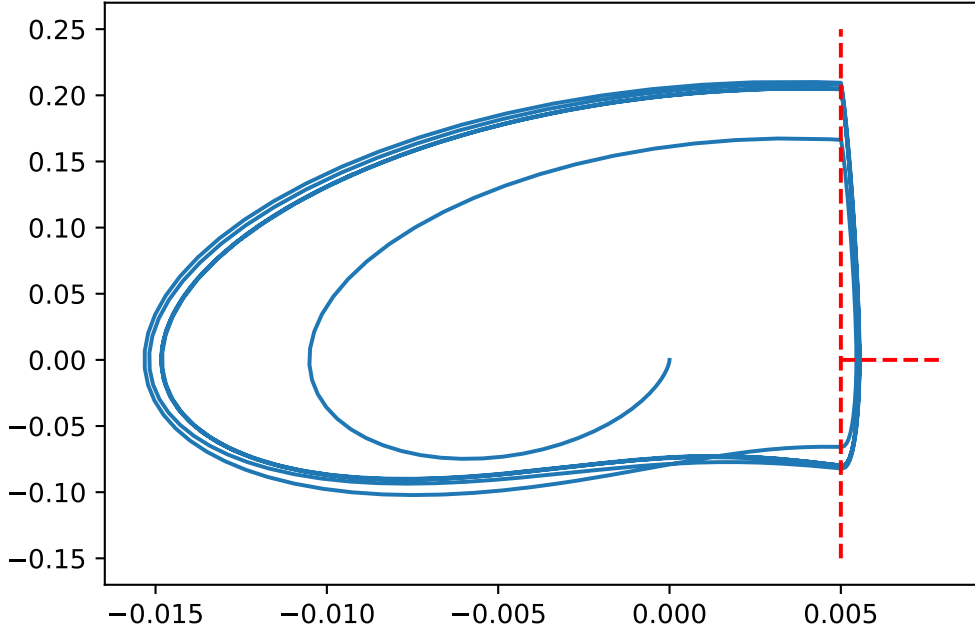


Figure 7: Phase diagram for structural pounding model

The discontinuity points (where the third derivative $x'''(t)$ is not continuous) lie onto the red dashed lines.

A Python code to integrate this problem with `disode45` can be

```
import numpy as np
import matplotlib.pyplot as plt
from disode45 import *
#
```



```

# Definition of the vector field
#
def fun(t,y):
    k=210.125
    c=2.47e+6
    nu=0.005
    r=2*np.sin(14*t)
    if y[0]>nu:
        if y[1]>0:
            u=c*(y[0]-nu)**(1.5) + 1.98*np.sqrt(2*c*np.sqrt(y[0]-nu))*y[1]
        else:
            u=c*(y[0]-nu)**(1.5)
    else:
        u=0;
    f=np.array([y[1], (-4.1*y[1]-k*y[0]-u-r)/2])
    return f

#
# Definition of the switching surface
#
def gfun(t,y):
    if y[0]<0.005:
        g=np.array([y[0]-0.005, 1])
    else:
        g=np.array([y[0]-0.005, y[1]])
    isterminal=[0,0]
    direction=[0,-1]
    return g,isterminal,direction

#
# Call to disode45
#
options=disodeset('Refine',10)
y0 = [0, 0]
tout,yout,tdis,ydis,idis,stats = disode45(fun,
                                           gfun,[0,3], y0,options)

#
# display of the results
#
print("\n tdis=",tdis)
print("\n ydis=",ydis)
print("\n idis=",idis)

```

Once the integration of this problem concludes, the vectors tdis, ydis and idis contain the following data

```

tdis= [0.40067767 0.40710588 0.41725649 0.83848014
       0.84463362 0.85438381 1.28186935 1.28803895

```

```

1.29782767 1.73074653 1.73693483 1.74675348
2.17989929 2.18608807 2.19590666 2.62872288
2.63491055 2.64472734]

```

```

ydis= [[ 5.00000000e-03  1.66374533e-01]
        [ 5.45479298e-03  4.55625121e-15]
        [ 5.00000000e-03 -6.57687148e-02]
        [ 5.00000000e-03  2.09451599e-01]
        [ 5.54777325e-03 -8.69096461e-16]
        [ 5.00000000e-03 -8.20121400e-02]
        [ 5.00000000e-03  2.07868890e-01]
        [ 5.54495793e-03 -2.31238639e-15]
        [ 5.00000000e-03 -8.10961005e-02]
        [ 5.00000000e-03  2.04800702e-01]
        [ 5.53853530e-03  3.25954541e-15]
        [ 5.00000000e-03 -7.98930610e-02]
        [ 5.00000000e-03  2.04657416e-01]
        [ 5.53819588e-03 -4.29170588e-15]
        [ 5.00000000e-03 -7.98498726e-02]
        [ 5.00000000e-03  2.04832128e-01]
        [ 5.53855549e-03  6.19816698e-15]
        [ 5.00000000e-03 -7.99182691e-02]]

```

```

idis= [1 2 1 1 2 1 1 2 1 1 2 1 1 2 1 1 2 1]

```

The plot in Figure 7 (without legends) can be obtained with the Python order

```

#
# Plot of the figures
#
plt.plot(yout[:,0], yout[:,1],[0.005,0.005],[-0.15,0.25], 'r--',[0.005, 0.008],[0,0], 'r-')
plt.show()

```

Example 5 A bouncing ball model (ODE with impulses) is governed by the equation

$$x'' = -9.8.$$

Starting from a height $x(0) = x_0$ with velocity $x'(0) = x'_0$, the ball falls and when it attains the floor $x(t_d) = 0$, with velocity $x'(t_d) = x'_d$, the integration must be restarted with initial conditions $x(t_d) = 0$, $x'(t_d) = -0.9 x'_d$ where the factor 0.9 represents the lost of energy.

Expressed as a first order system with two components $y_1(t) = x(t)$, $y_2(t) = x'(t)$, we have

$$y' = \begin{pmatrix} y'_1 \\ y'_2 \end{pmatrix} = \begin{pmatrix} y_2 \\ -9.8 \end{pmatrix} = f(t, y)$$

Here the switching surface is clearly $g(y) = y_1$.

In this case the discontinuity affects the solution, and the jump of the state must be provided in some way by the user. In the code this is done by means of an external function `actionatswitch(t,y)` that from the current state y_d provides the new state y_d^* .

A Python code to integrate this problem with `disode45` can be

```
import numpy as np
import matplotlib.pyplot as plt
from disode45 import *

#
# Definition of the vector field
#
def fun(t,y):
    ydot=np.array([y[1], -9.8])
    return ydot

#
# Definition of the switching surface
#
def gfun(t,y):
    g=y[0]
    isterminal=-1    # Call to gswitch when a switching point is found
    direction=-1
    return g,isterminal,direction

#
# Output switch function
#
def actionatswitch(t,y):
    ysw=np.array([0, -0.9*y[1]])
    return ysw

#
# Call to disode45
#
options=disodeset('AbsTol',1.e-4,'RelTol',1.e-4,
                  'Refine',10, 'ActionSwitch', actionatswitch);
y0=np.array([10, 0])
tout,yout,tdis,ydis,idis,stats = disode45(fun,
                                           gfun, [0,20], y0, options);

#
# display of the results
#
print("\n tdis=",tdis)
```

```

print("\n ydis=",ydis)
print("\n idis=",idis)
#

```

Once the integration of this problem concludes, the vectors `tdis`, `ydis` and `idis` contain the data corresponding to 13 switching points.

```

tdis= [ 1.42857143  4.          6.31428571  8.39714286
        10.27171429 11.95882857 13.47723143 14.843794
        16.07370031 17.180616   18.17684011 19.07344181
        19.88038335]

```

```

ydis= [[ 4.44089210e-15 -1.40000000e+01]
        [ 8.88178420e-15 -1.26000000e+01]
        [ 3.28626015e-14 -1.13400000e+01]
        [ 7.99360578e-15 -1.02060000e+01]
        [ 2.75335310e-14 -9.18540000e+00]
        [ 7.54951657e-15 -8.26686000e+00]
        [ 1.55431223e-14 -7.44017400e+00]
        [-8.21565038e-15 -6.69615660e+00]
        [ 2.25375274e-14 -6.02654094e+00]
        [ 1.24344979e-14 -5.42388685e+00]
        [-9.54791801e-15 -4.88149816e+00]
        [-1.68753900e-14 -4.39334835e+00]
        [ 1.04360964e-14 -3.95401351e+00]]

```

```

idis= [1 1 1 1 1 1 1 1 1 1 1 1 1]

```

Example 6 A heating model (problems with a switching vector field). The variation of the temperature in a room is supposed to be linear on the difference $x(t) - T_{\text{ext}}$ between the current temperature $x(t)$ and the external temperature T_{ext} . Initially, if the temperature $x(0)$ is lower than a maximum temperature T_{max} , the heating is on, a constant external heat source u is acting and the temperature satisfies the equation

$$x' = -K \cdot (y - T_{\text{ext}}) + u,$$

until the temperature attains T_{max} . At that moment the heat source u is set off and the differential equation changes to

$$x' = -K \cdot (y - T_{\text{ext}}).$$

The heat source is set on again when $x(t) = T_{\text{min}}$.

Note that in this example the vector field itself is modified from $f(t, x)$ to a different one $\hat{f}(t, x)$ when the solution attains certain point t_d satisfying

a condition $g(t_d, x(t_d)) = 0$. For the same point (t, x) , the vector field $f(t, x)$ can not be equal to $\hat{f}(t, x)$.

In order to solve this problem with DISODE45, it must be transformed, by adding an additional equation, into an equivalent problem that can be treated as an ODE with impulses. Defining $y(t) = (y_1(t), y_2(t))^T \equiv (x(t), y_2(t))^T$, where $y_2(t)$ is going to be a constant, the heating can be modelled by

$$\begin{aligned} y_1' &= \begin{cases} -K \cdot (y_1 - T_{\text{ext}}) + u & \text{if } y_2(t) = 1, \\ -K \cdot (y_1 - T_{\text{ext}}) & \text{if } y_2(t) = -1, \end{cases} \\ y_2' &= 0, \\ y_1(0) = x(0) = x_0, \quad y_2(0) &= \begin{cases} 1 & \text{if } x_0 < T_{\text{max}}, \\ -1 & \text{if } x_0 \geq T_{\text{max}}. \end{cases} \end{aligned}$$

There are two switching surfaces $g_1(y) = y_1 - T_{\text{max}}$ and $g_2(y) = y_1 - T_{\text{min}}$. When the solution attains the first one from temperature $x(t) < T_{\text{max}}$, or the second one from temperature $x(t) > T_{\text{min}}$, the sign of the variable $y_2(t_d)$ is changed.

A Python code to integrate this problem with `disode45` can be

```
import numpy as np
import matplotlib.pyplot as plt
from disode45 import *

#
# Definition of the vector field
#
def fun(t,y):
    if y[1]==-1:
        ydot=np.array([-0.1*(y[0]-18), 0])
    else:
        ydot=np.array([-0.1*(y[0]-18)+2, 0])
    return ydot

#
# Definition of the switching surface
#
def gfun(t,y):
    g=[y[0]-23.5, y[0]-22]
    isterminal=[-1,-1] # Call to actionatswitch when found
    direction=[1,-1] # From negative to positive the first one
    return g,isterminal,direction

#
# Output switch function
#
```

```

def actionatswitch(t,y):
    if y[1]==1:
        yswitch=[y[0], -1]
    else:
        yswitch=[y[0], 1];
    return yswitch

#
# Call to disode45
#
options=disodeset('AbsTol',1.e-4,'RelTol',1.e-4,
                  'ActionSwitch', actionatswitch);
y0=np.array([15, 1])
tout,yout,tdis,ydis,idis,stats = disode45(fun,
                                           gfun, [0,20], y0, options);

#
# display of the results
#
print("\n tdis=",tdis)
print("\n ydis=",ydis)
print("\n idis=",idis)

```

Once the integration of this problem concludes, the vectors `tdis`, `ydis` and `idis` contain the data corresponding to 13 switching points.

```

tdis= [ 4.61345568  7.79799289  8.78239366 11.96693127
       12.951332   16.13586921 17.12026994]

ydis= [[23.5  1. ]
       [22.  -1. ]
       [23.5  1. ]
       [22.  -1. ]
       [23.5  1. ]
       [22.  -1. ]
       [23.5  1. ]]

idis= [1 1 1 1 1 1 1]

```

References

- [1] V. Acary and B. Brogliato, *Numerical Methods for Nonsmooth Dynamical Systems* (Lecture Notes in Applied and Computational Mechanics, Springer–Verlag, Berlin, 2008).
- [2] M. Calvo, J.I. Montijano and L. Rández, *DISODE45, A Python Runge-Kutta solver for Piecewise Smooth IVPs of Filippov type*, to appear in ACM TOMS.
- [3] R. Jankovski, *Non-linear viscoelastic modelling of earthquake-induced structural pounding*, Earthq. Eng. Struct. Dyn. 34(2005), pp. 595-611.
- [4] A.C.J. Luo, *Discontinuous Dynamical Systems on Time-varying Domains* (Non-linear Physical Science, Springer–Verlag, Berlin, 2009).